

实验报告：MiniDraw 图形扩展（Ellipse / Polygon / Freehand）

课程名称：计算机图形学

实验题目：基于 Framework2D 的 MiniDraw 图形扩展

姓名：李佩优 学号：PB23000270 日期：20260308

一、实验目标

1. **理解框架结构**：分析 MiniDraw 项目中的 **Canvas** 类、**Shape** 基类以及鼠标事件处理机制。
 2. **功能扩展**：
 - 添加椭圆（**Ellipse**）绘制功能，支持鼠标拖拽实时预览。
 - 添加多边形（**Polygon**）绘制功能，支持左键加点、右键结束。
 - （选做）添加自由曲线（**Freehand**）绘制功能。
 3. **掌握面向对象技术**：通过继承 **Shape** 基类实现多态绘制，体会封装和多态的应用。
-

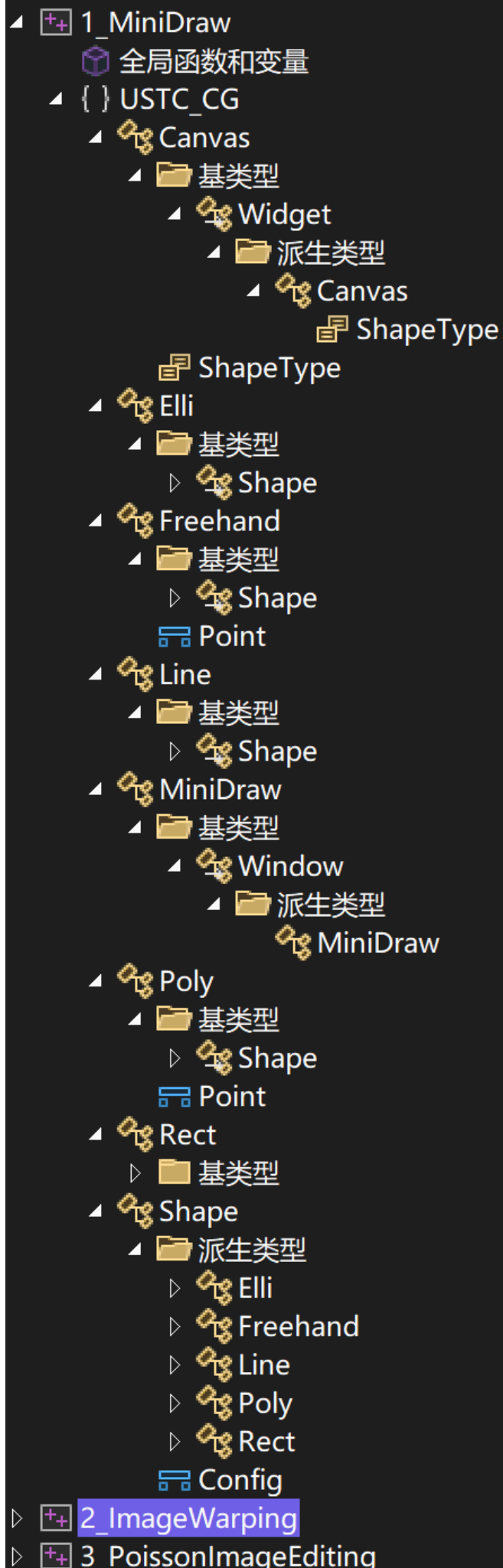
二、原有框架分析

2.1 框架整体结构

MiniDraw 基于 Framework2D 和 Dear ImGui 构建，主要模块如下：

- **MiniDrawWindow**：主窗口，负责菜单栏、按钮的绘制，响应用户点击切换图形类型。
- **Canvas**：画布控件，管理图形列表（**shapes_**）、当前正在绘制的图形（**current_shape_**）、绘制状态（**draw_status_**）以及鼠标事件。
- **Shape 类及其派生类**：所有图形的抽象基类，定义了纯虚函数 **draw()** 和 **update()**。现有派生类包括 **Line**、**Rect**。
- **事件处理流程**：
 - 鼠标按下：创建对应图形对象，存入 **current_shape_**，进入绘制状态。
 - 鼠标移动：调用 **current_shape_->update()** 实时更新图形参数。
 - 鼠标释放（或右键点击）：将 **current_shape_** 加入 **shapes_** 列表，退出绘制状态。

2.2 关键类关系



```
▷  ALL_BUILD
▷  common
▷  demo
▷  demo_hello_world
▷  glad
▷  glfw
▷  imgui
▷  INSTALL
▷  uninstall
▷  update_mappings
▷  ZERO_CHECK
```

`Shape` 为抽象基类，`Line`、`Rect` 继承自 `Shape`。`Canvas` 包含 `std::vector<std::shared_ptr<Shape>>` 存储已完成的图形，以及一个 `std::shared_ptr<Shape> current_shape_` 表示正在绘制的图形。

2.3 框架预留的扩展点

- 通过添加新的 `Shape` 派生类实现新图形。
- 通过修改 `Canvas::shape_type_` 和 `mouse_click_event()` 中的 `switch` 分支，支持新图形类型的构造。
- 通过添加新的 `ImGui` 按钮，让用户选择图形类型。

三、设计与实现

3.1 椭圆（Ellipse）的添加

3.1.1 设计思路

- **遵循框架约定**：创建 `Ellipse` 类继承自 `Shape`，存储左上角 (`xmin, ymin`) 和右下角 (`xmax, ymax`)，实现 `draw()` 和 `update()`。
- **复用现有交互**：完全沿用直线和矩形的“按下-移动-再次按下”交互模式，只需在 `mouse_click_event()` 中增加 `kEllipse` 分支构造 `Ellipse` 对象。

3.2 多边形（Polygon）的添加

3.2.1 设计思路

- **交互设计**：左键单击添加顶点，移动鼠标时实时显示当前线段，再次单击起点结束绘制(使用距离判断)。
- **数据结构**：`Polygon` 类维护两个 `float` 存储起点位置；创建 `Point` 结构集成点坐标；一个 `std::vector<Point>` 存储中间点坐标。
- **绘制**：在 `draw` 中遍历顶点列表，用 `AddLine` 依次连接各点，倒数第二个点直接与起点连接。

3.3 自由曲线（Freehand）的实现

3.3.1 设计思路

- **交互设计**：左键单击添加，移动鼠标时每隔一小段距离 ($\sqrt{50}$) 绘制一段小直线，直至鼠标释放结束绘制。
- **数据结构**：同椭圆，创建Point结构集成点坐标,Freehand 类维护维护一个std::vector<Point> 存储顶点坐标。
- **绘制**：在 draw 中遍历顶点列表，用 AddLine 依次连接各点。

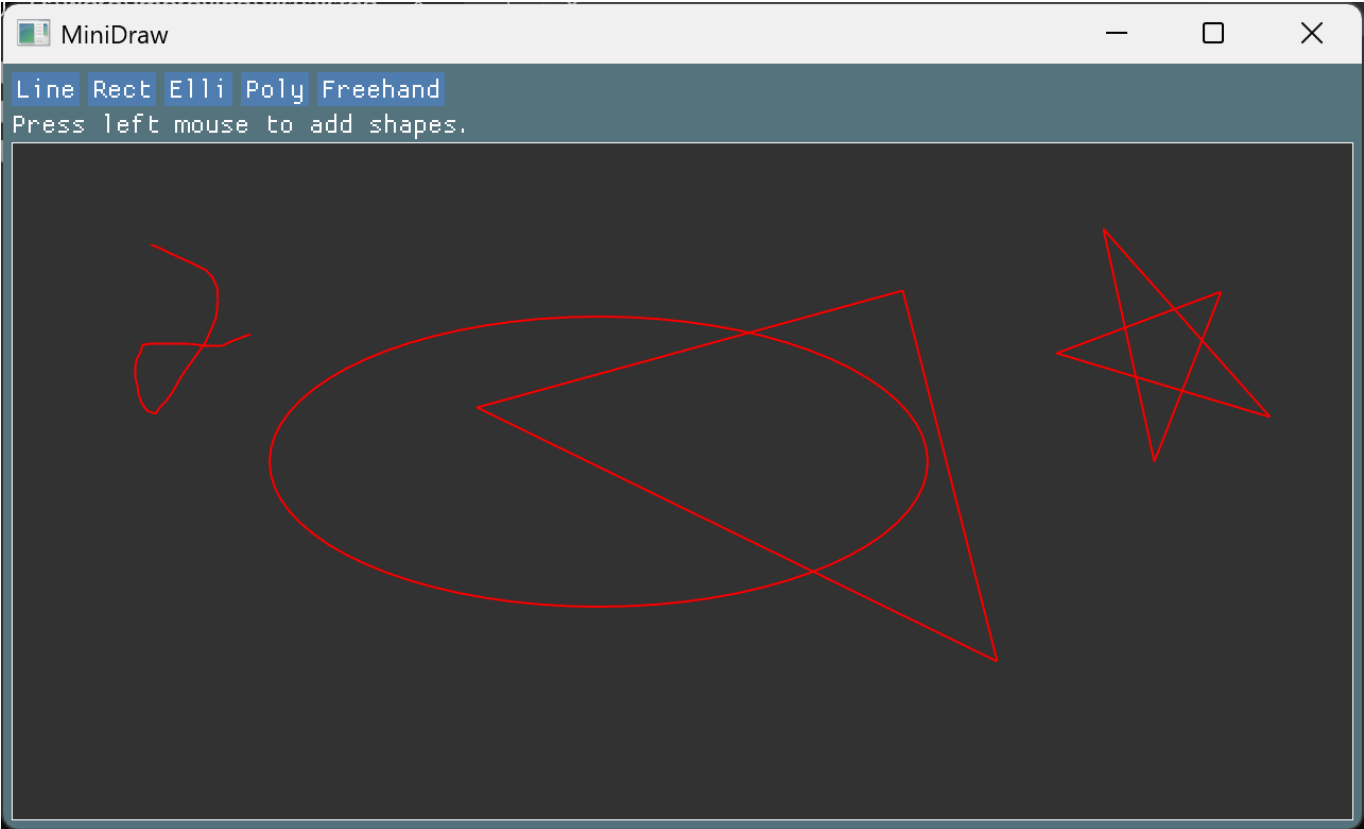
四、测试与验证

4.1 测试环境

- 操作系统：Windows 10
- 编译器：MSVC 2019
- 依赖库：Framework2D (内置 ImGui 1.89)

4.2 测试用例

功能	操作步骤	预期结果	实际结果
椭圆绘制	点击“Ellipse”按钮，在画布上按下鼠标左键并拖拽，释放左键	出现椭圆，且拖拽过程中实时显示椭圆轮廓	符合预期
多边形绘制	点击“Polygon”按钮，左键依次点击添加顶点，右键结束	显示多边形，最后一个点与第一个点自动连接	符合预期
切换图形	在绘制椭圆后点击“Rect”按钮，继续绘制矩形	矩形正常绘制，椭圆保持不变	符合预期



五、总结与反思

5.1 实验完成度

- 完成了椭圆、多边形、自由曲线的绘制功能。
- 按钮切换正常，所有新图形均能正确保存到 `shapes_` 列表中。

5.2 对框架的理解

- 通过继承 `Shape` 并重写 `draw` 和 `update`，实现了多态绘制，深刻体会了**开闭原则**（对扩展开放，对修改封闭）。
- 鼠标事件的处理流程清晰，将绘制状态与图形数据分离，降低了耦合度。

5.3 展望

- 多边形绘制过程中，可添加“撤销上一个点”的功能（如右键菜单）。
- 框架中的 `Config` 参数可以扩展（如颜色、线型），但当前未完全利用，后续可优化。
- 在多边形绘制功能当中没有区分是否为凸多边形，以及边是否交叉，有待改进。
- 在自由绘制功能当中未对点的总数限制，如果一直不停可能非常大（？）
- 可以在添加功能的基础上加入选中功能，或修改功能，